TUTORIAL 3

Intermediate IC Design Flow

SUBMITTED BY

Kamrul Hasan Ovi
Matriculation Number: 6335212

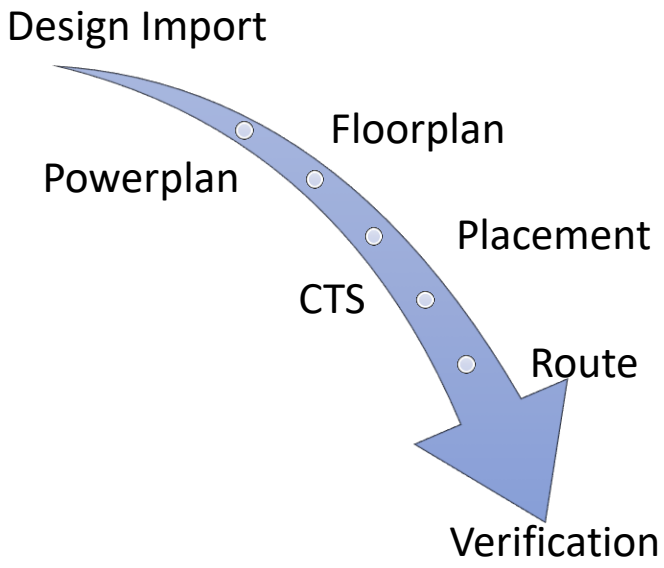
1 Goals

The goal of this tutorial to learn intermediate level design flow for creating an IC. In this lab, we covered Macro generation, synthesis and detailed backend design. The module we developed is a `4×4 2D mesh NOC`.

The design steps include

First, the Processing Element (PE) was created as a simple RTL block representing a node in the NoC. This PE was synthesized into a gate-level netlist and taken through placement and routing to form a complete physical macro. After that, the macro views were exported as LIB, LEF, and DEF files so it could be reused in higher-level design.

Next, the top-level NoC design was built by connecting multiple PE macros. The full RTL was synthesized using Genus to generate a complete gate-level netlist. After synthesis, the design moved into the physical design flow.



2 Macro Generation

In this section, we learnt how to generate the lef and lib files of a macro.

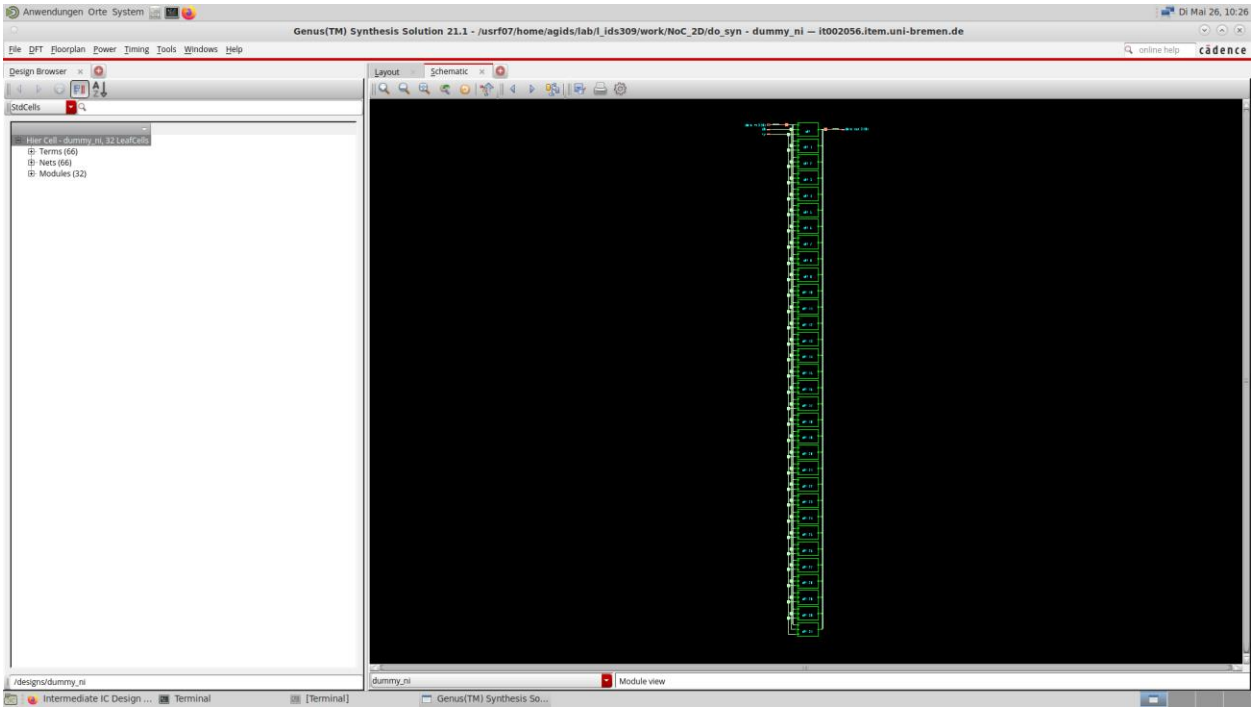
2.1 Synthesis (PE)

In the macro generation part, we synthesized our PE. For the purpose, we used a dummy block which represents the PEs in NoC.

Synthesis Process

- **Environment Setup** – Configured the `Cadence Genus` environment and loaded the standard cell library.

- **RTL Import** – Read the VHDL source files of the `dummy_ni` design.
- **Clock Gating and Elaboration** – Enabled clock gating for power reduction and elaborated the design to build its internal representation.
- **Constraint Definition** – Applied I/O, clock, and timing constraints, including clock period, uncertainty, latency, and output delays.
- **Logic Synthesis** – Performed generic synthesis, technology mapping, and optimization to generate the gate-level implementation.
- **Design Analysis** – Generated timing, area, gate count, clock-gating, and power reports.
- **Design Export** – Saved the synthesized netlist and timing constraint files for the physical design flow.



3 Place and Route of PE

3.1 Import Design

Inputs Required for import:

- **Netlist (.v or .vhd)** → It is the combination of sequential elements and their logical Connectivity. The netlist file name of this design is `Dummy_netlist.v` that we got from synthesis.
- **Synopsys Design Constraints (SDC)** →In the SDC file, Constraints related information are mentioned, related to design rule constraints, optimization constraints, timing constraints.
- **Liberty Timing File (.lib)** → it contains timing related information, for example cell type and Functionality, Delay Models. pin/ Cell timings and design rules, PVT conditions, power details. The library file for the design, `NangateOpenCellLibrary_typical_ccs.lib`, is used.

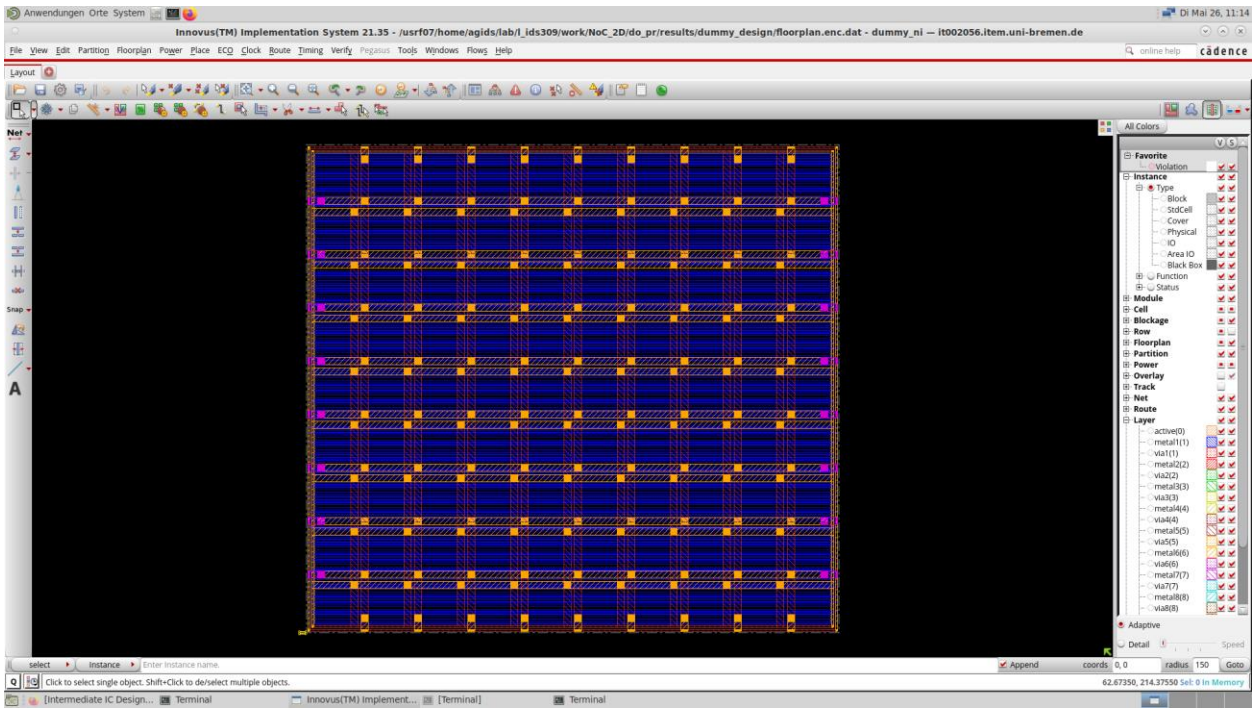
- **Library Exchange Format (LEF)** → LEF file contains physical information of the designs. LEF contains cell name, Shape, Size, orientation & class, Port/pin name, Direction and layout Geometrics, Obstruction/Blockages. For this design, we used the `NangateOpenCellLibrary.lef file`.
- **Multiple mode multiple corner file (MMMC.view)**

3.2 Floorplan

Steps in Floorplan

- Initialize with Chip & Core Aspect Ratio
- Initialize with Core Utilization
- Power Rings
- Pins /Pads Placement
- Macro Placement

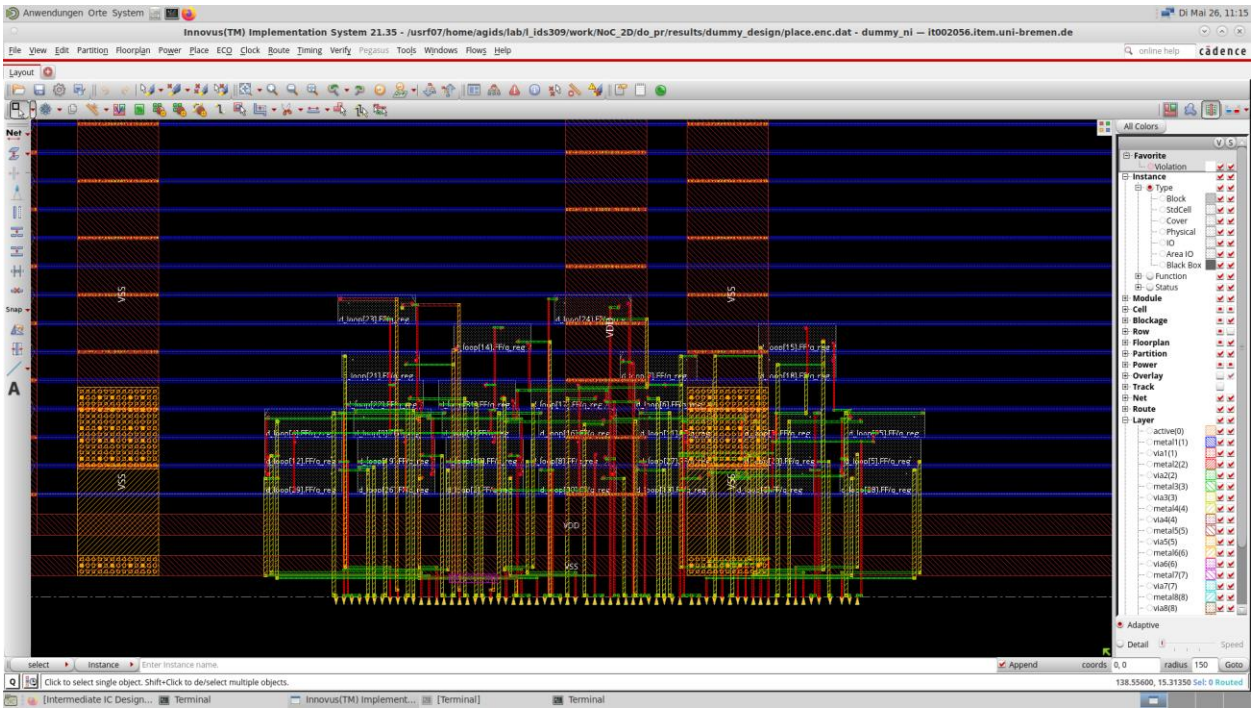
For the design, we defined a floorplan with die box size of 300*275 and the spacing, in micrometer, between die edge. We also specified the pin metal. For top and bottom, we used the metal 2 and for right and left we used the metal 3. Next, we used `globalNetConnect` command to connect PG pins to the specified global net, which is either a power or ground net. Then, we added power rings and stripes, where we specified the metal layer for ring and stripes. We used metal 5 and 6 for rings and stripes.



3.3 Placement

Placement is the process of placing standard cell in the design. The tools determine the location of each standard cell on the dies. The tool places these based on the algorithms which it used internally. Placement does not just place the standard cells available in the synthesized netlist. It also optimizes the design. Placement also determines the routability of design.

During the placement stage, all constraint mode was activated to generate a list of constraint mode. A high congestion effort setting was used.



Placement Outputs

- Timing report
- Design with all std cells placed in core area

3.4 CTS

Clock Tree Synthesis is one of the most important stages in PnR. The process of distributing the clock and balancing the load is called CTS. CTS is the process of insertion of buffers or inverters along the clock paths if ASIC design in order to achieve zero/minimum slew or balanced skew.

During the CTS stage, the ana1 analysis view was selected for both setup and hold timing analysis. Timing analysis was configured with on-chip variation and common path pessimism Removal enables to improve timing accuracy. A non-default routing rule with increased wire spacing was defined and applied to the clock tree routing. Different types of routing are applied, leaf_rule, trunk_rule a top_rule, to improve signal integrity. Clock tree optimization parameters, including target maximum transition, target skew, maximum fanout, and allowed clock buffer and inverter

cells, were specified. A clock tree specification file was then generated and used to guide the clock tree synthesis process.

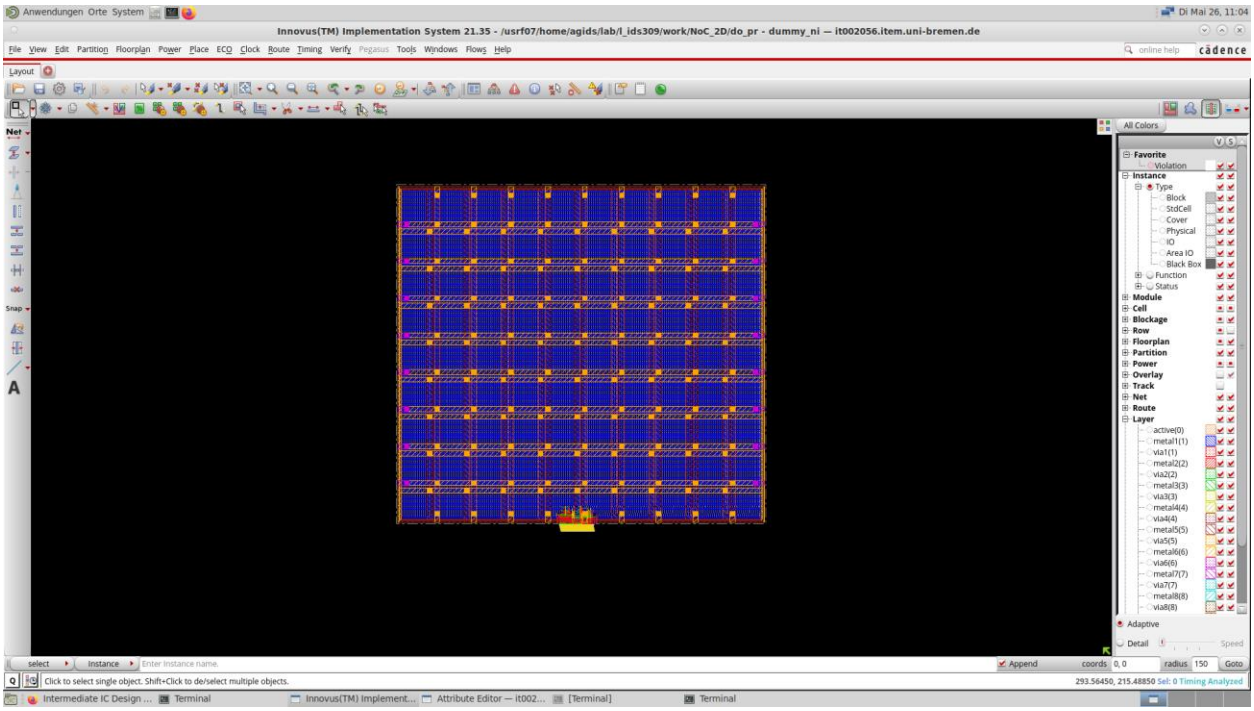
CTS output

- Timing report
- Skew report
- Congestion report

3.5 Routing

Making physical connections between signal pins and the standard cells are called routing. Electrical connections using metal layers and vias are created in the layout, defined by the logical connections present in the netlist. After CTS, we have information of all the placed cells and clock tree buffers/inverters and I/O pins, the tool relies on this information to electrically complete all connection defined in the netlist.

During the routing stage, propagated clocks were enabled so that timing was calculated using the actual clock network delays. Signal-integrity-driven and timing-driven routing were applied to improve timing quality. After global and detailed routing, filler cells were added and any remaining DRC violations were fixed using ECO routing. Finally, post-route timing analysis was done, the design was exported to DEF format, and the final routed design was saved.



Outputs of Routing

- Timing report
- Geometric layouts of all nets

3.5.1 Extracting LEF, DEF, and timing models

After completing the design, we dump out the LEF, DEF and Lib file.

3.6 Synthesis

In this process, we synthesized the top module along with the generated dummy PE block. The synthesis flow is the same as described in `Section 2.1`. After completing synthesis, we proceeded to the placement and routing stage.

3.7 Place and Route

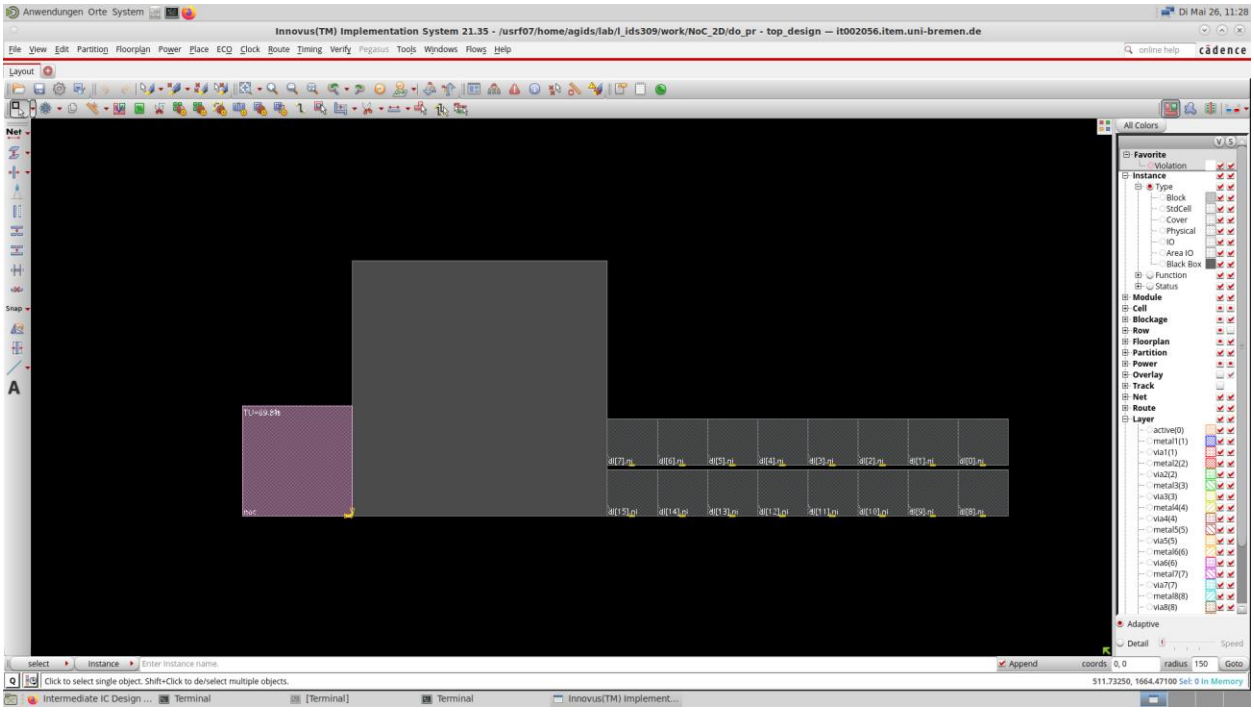
3.7.1 Setup the environment

At first, we setup the `innovus` environment to run our design.

3.7.2 Read the design

Before we read our design, we set the `init_design_uniquify` 1 to ensure that all instances and nets in the design have unique names, avoid conflicts and potential issues during the stages of the physical design flow.

After we load the synthesized netlist, LEF files, and MMMC view. Power and ground nets were defined. after initialization, the full hierarchy was loaded successfully.



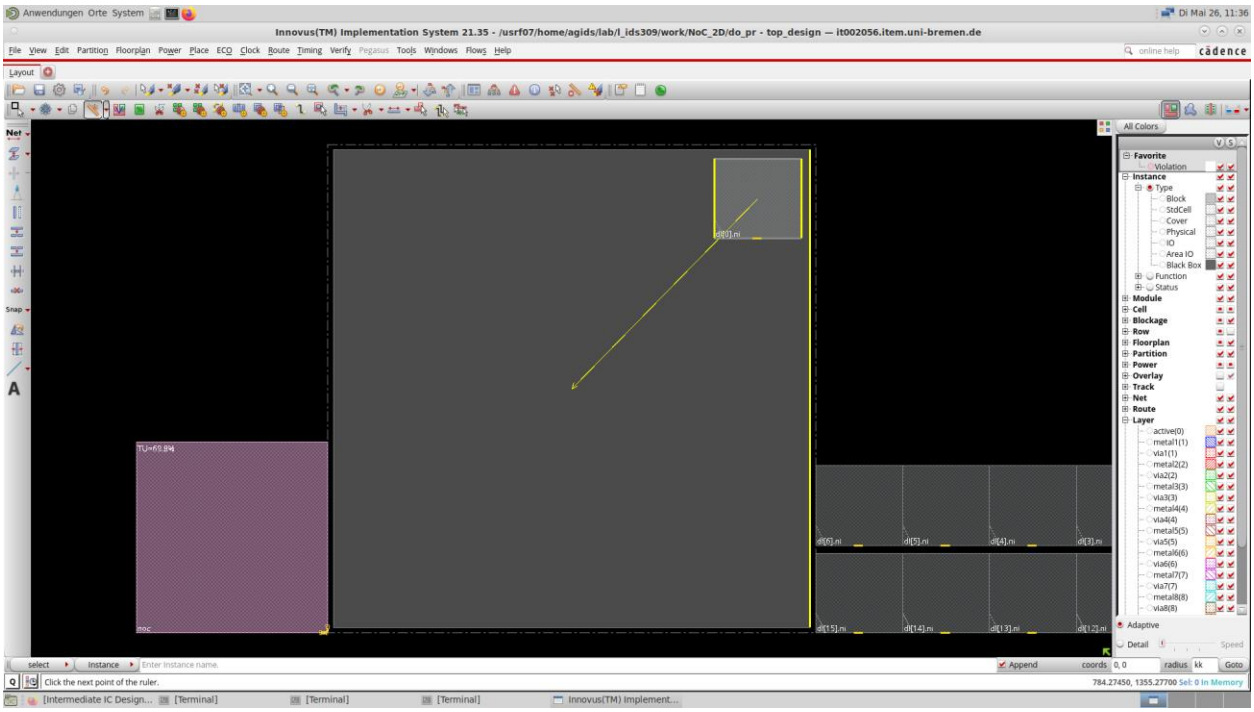
3.7.3 Floorplaning

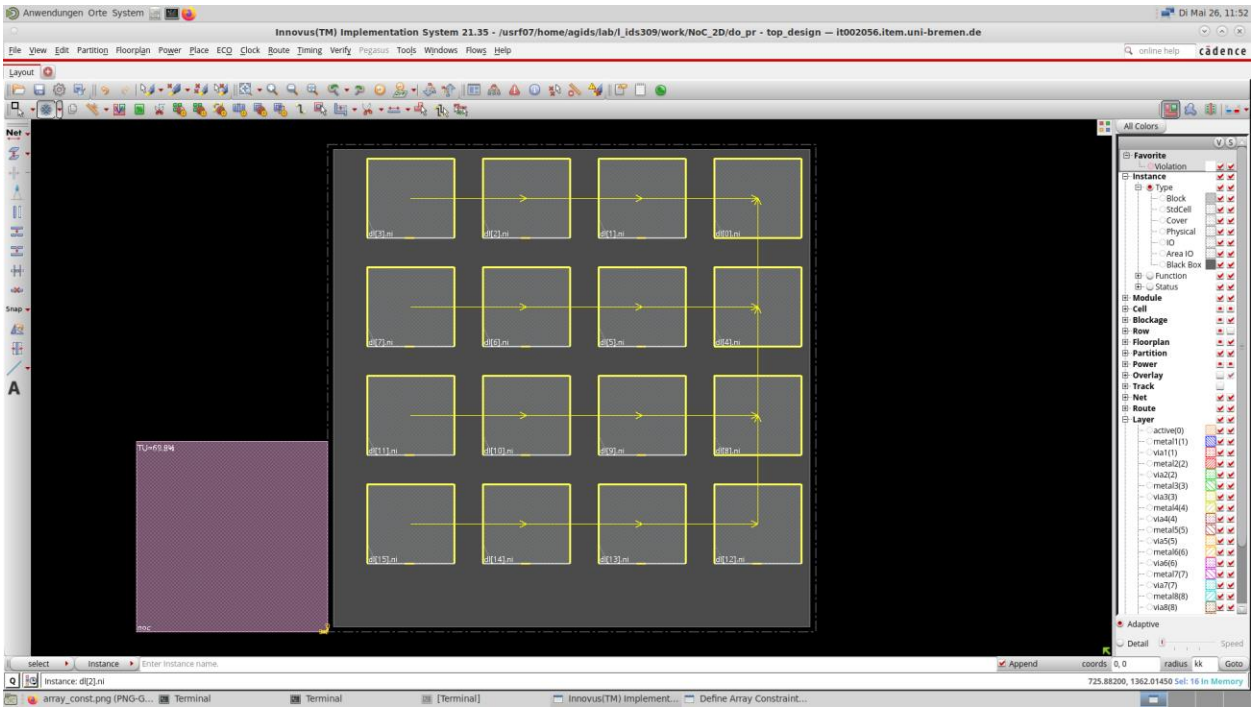
First, the I/O pins were assigned to specific metal layers and placed on different sides of the chip. Then, the floorplan was created by defining the chip size, shape, and the space around the core area.

3.7.3.1 Macro Placement

In this step, we placed the macros in a simple and systematic way using relative floorplan. First, we selected one macro (`dl[0].ni`) as a reference point. Then we tried relative floorplan so that other macros can be placed based on this reference instead of fixed positions.

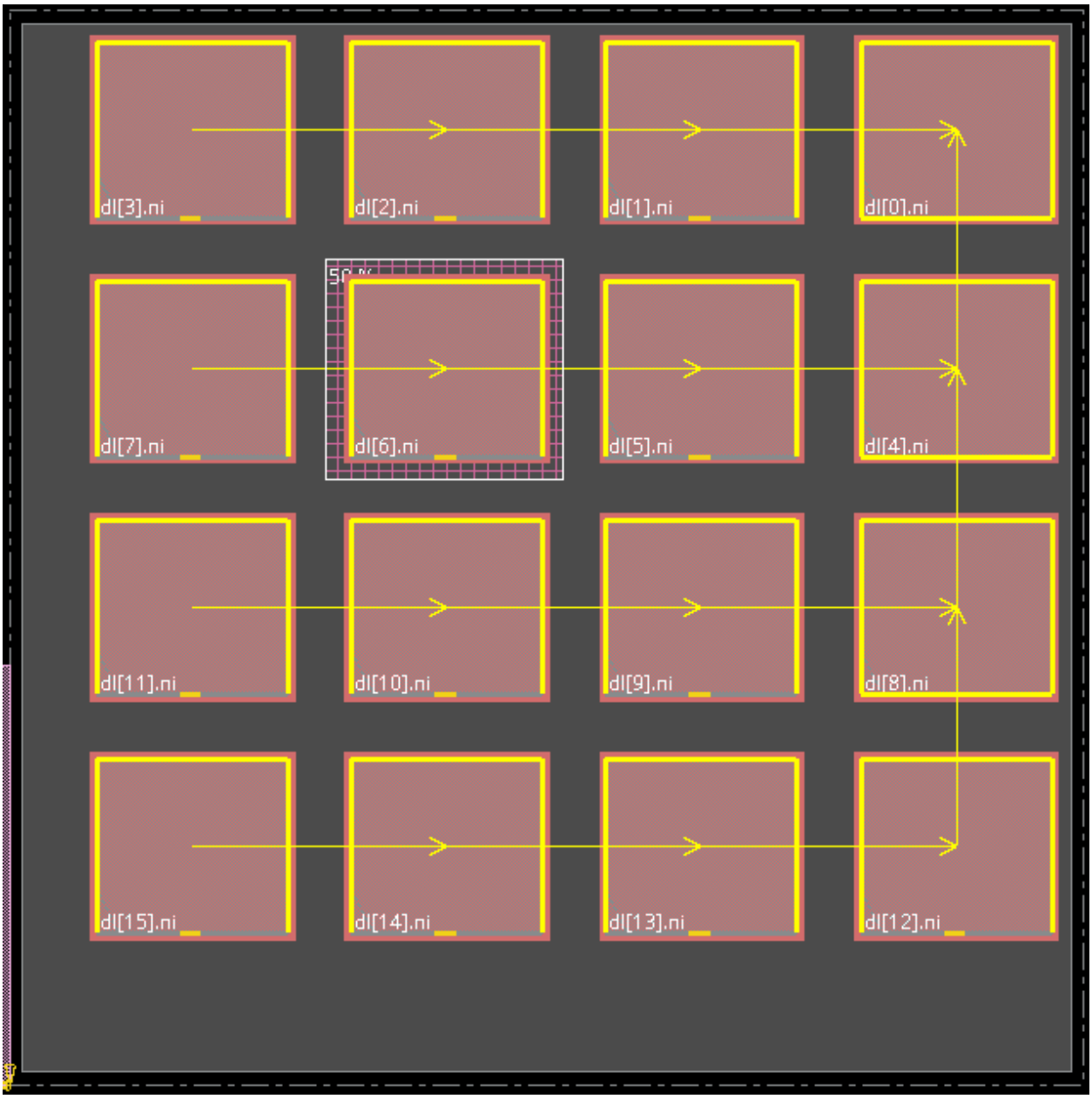
After checking this, we removed the relative floorplan settings. Then we selected all 16 macros (`dl[0].ni to dl[15].ni`) and arranged them using array constraint to make a proper 4×4 structure for the NoC. In the end, we cleared all selections and finished the macro placement setup for the next steps.





3.7.3.2 Placement and routing blockage

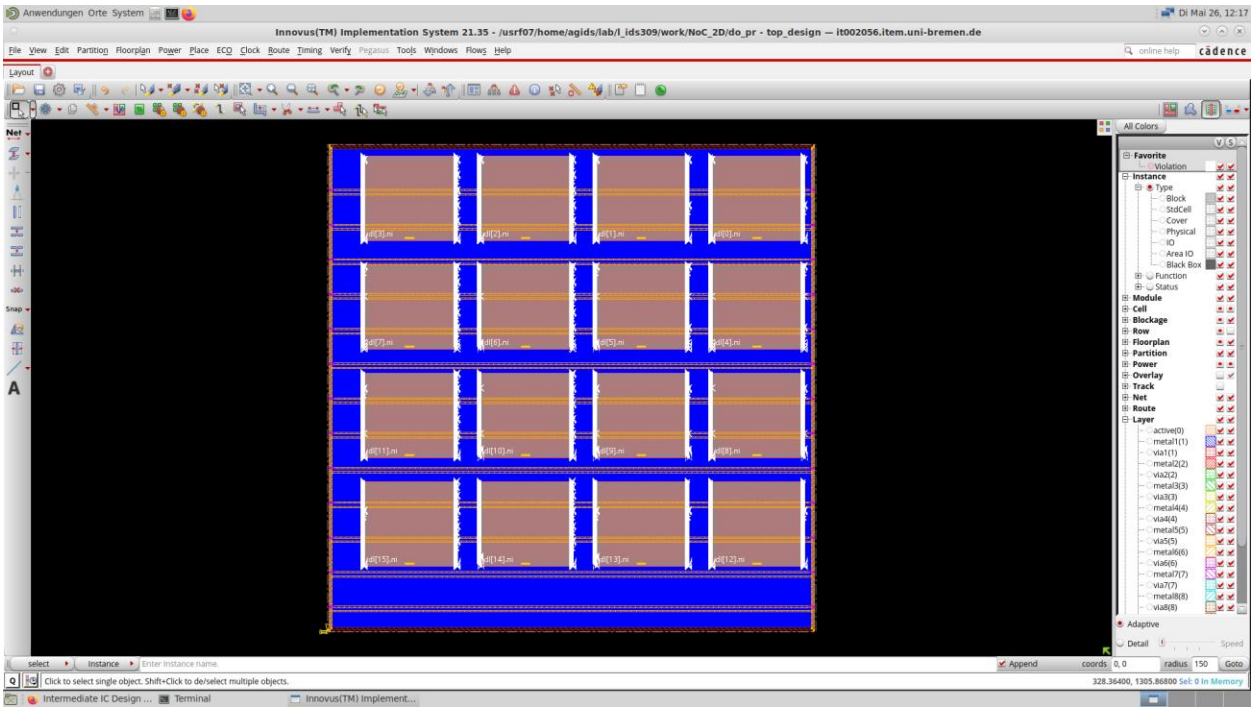
In this step, we added placement halos and blockages around the macros to control where standard cells can be placed. First, a halo was added around the blocks so that no cells can be placed too close to the macros.



3.7.3.3 Power Routing

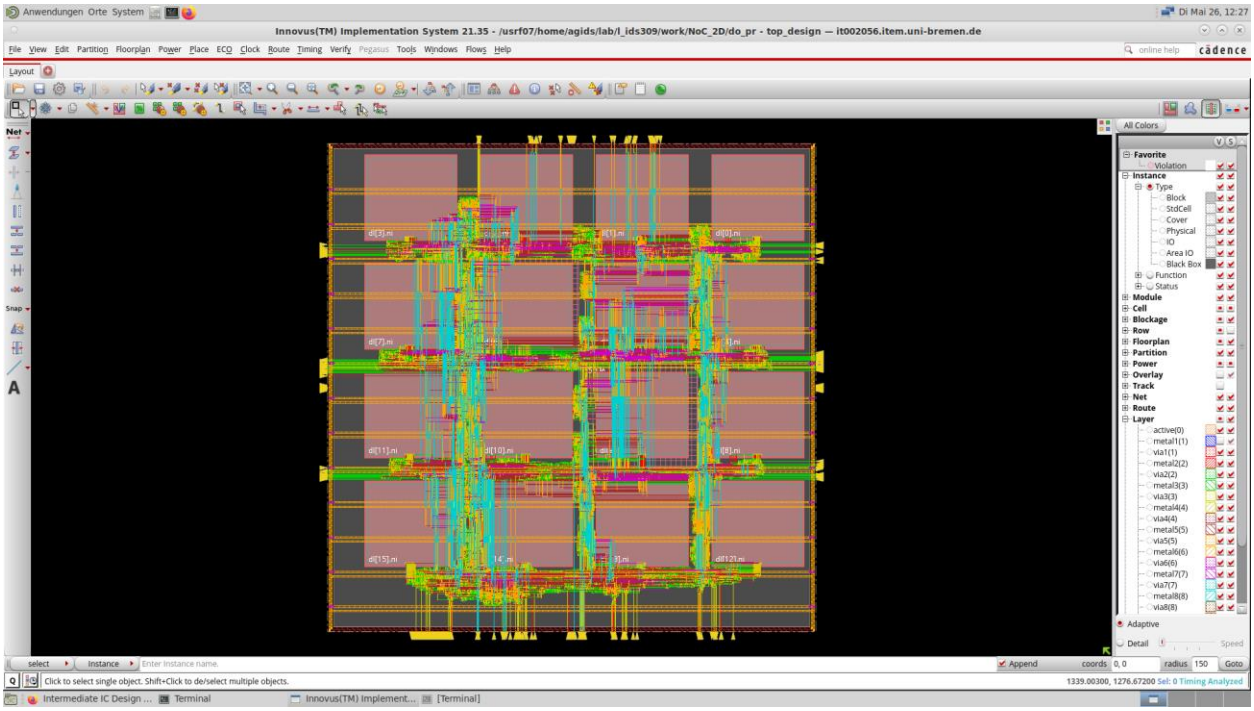
In this step, we first connected the global power and ground nets for the whole design by assigning VDD and VSS to all blocks and pins. After that, power planning was done to build a proper power distribution network. Power rings were added around the core area, and power stripes were created in both vertical and horizontal directions to distribute power evenly across the chip. Unfortunately, my vertical stripes are not connected for unknown reason. I continued the further steps with existing power plan.

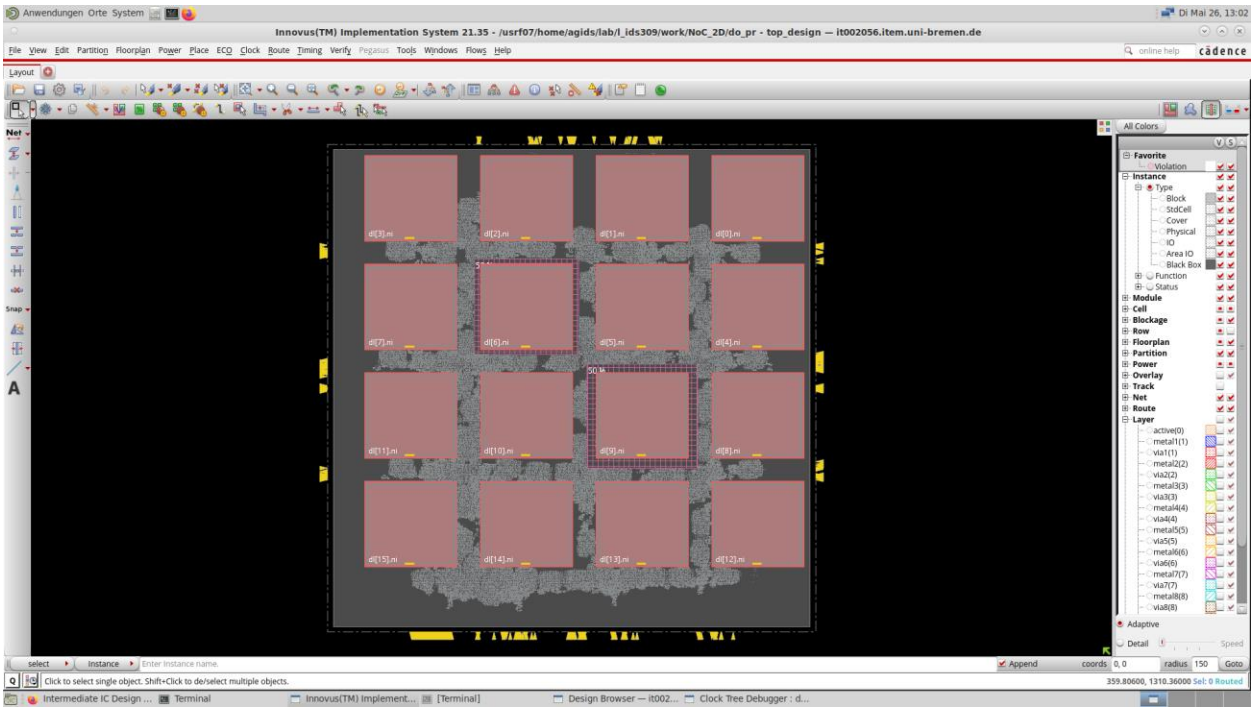
After completing the power grid, the design was checked for DRC issues to make sure there are no violations. Finally, floorplan reports were generated and the design was saved for the next stage.



3.7.4 Placement of Standard Cells

After floor planning, we placed all the standard cells. Then, the placement optimization is performed, where all standard cells are placed inside the floorplan area while trying to improve timing and reduce congestion.



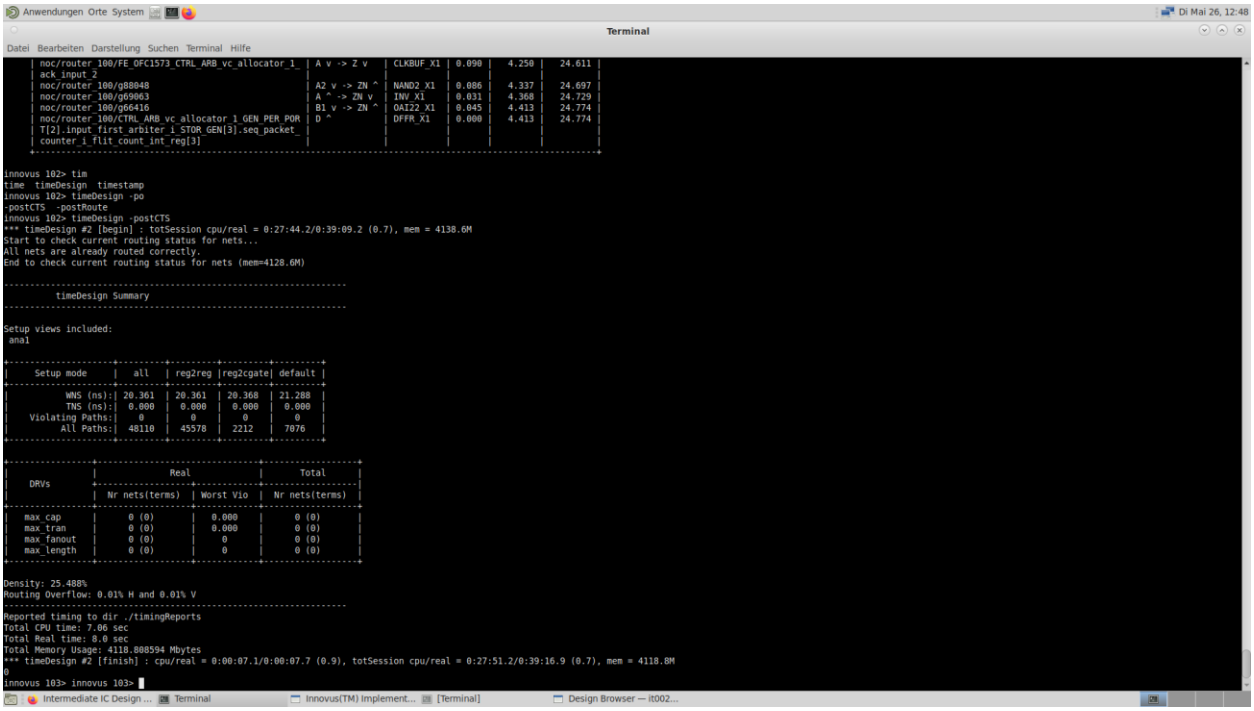


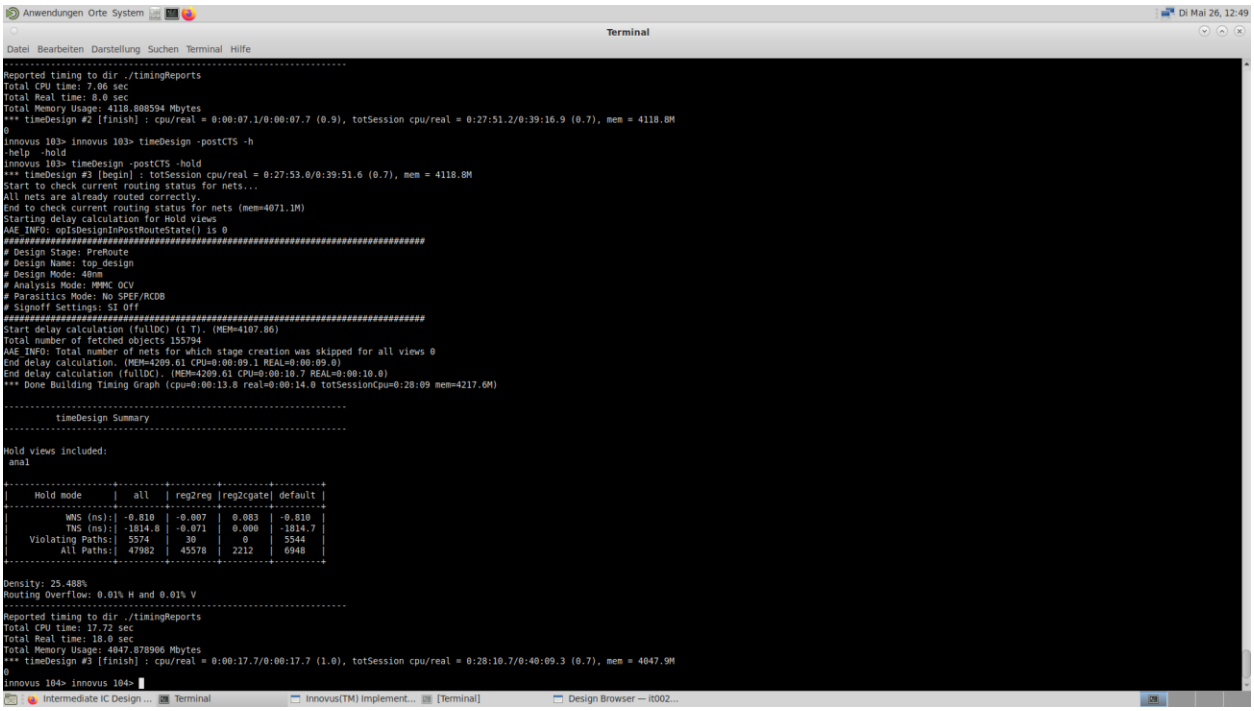
3.7.5 Clock Tree Synthesis

In this section, clock tree synthesis (CTS) was performed to build a balanced clock network for the design. First, timing analysis mode was set to on-chip variation with CPPR enabled to reduce pessimism in setup and hold analysis. Non-default routing rules were defined to strengthen and protect clock nets, and the clock tree structure was divided into leaf, trunk, and top routing layers with proper shielding.

Next, CTS constraints were configured, including target skew, maximum transition, fanout limits, and the selection of buffer and inverter cells for clock tree construction. The useful skew method was enabled. After setting all required properties, the clock tree was synthesized using `CCOpt`, and the design was optimized to improve overall timing performance, followed by post-CTS timing checks to verify setup and hold constraints.

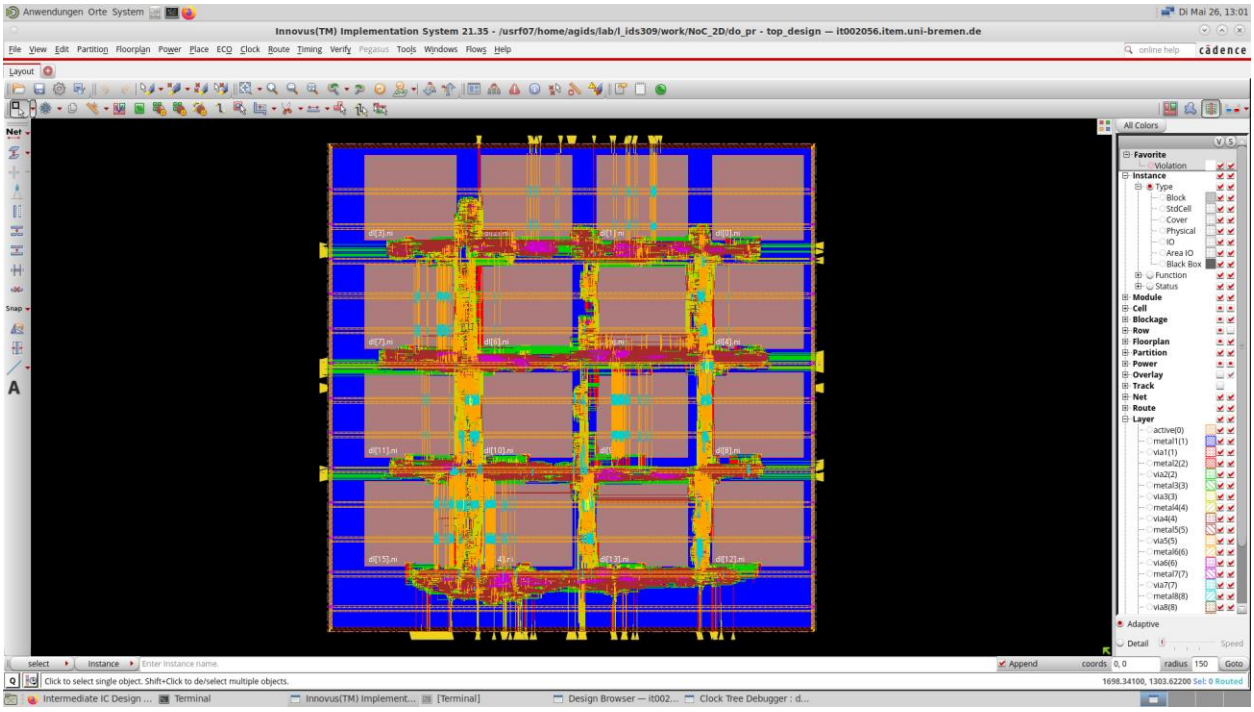

Clock Tree Debugger : default — it002056.item.uni-bremen.de
View Visibility Color by Help
cadence

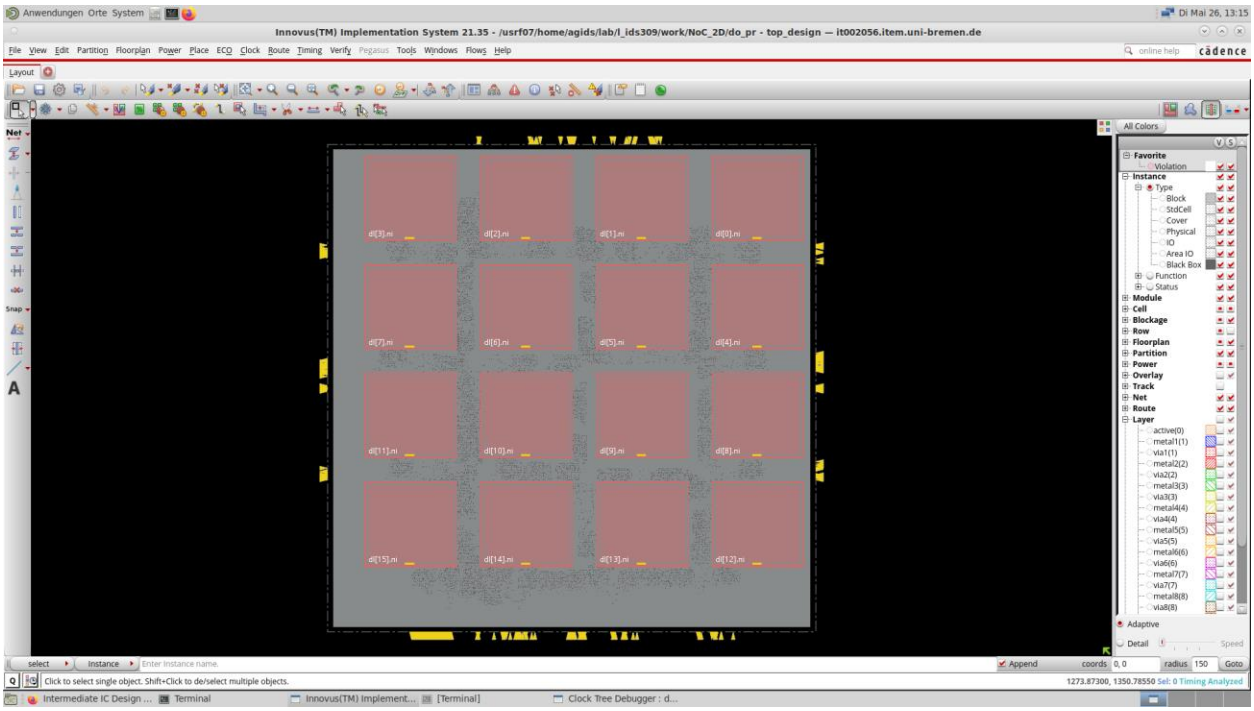

Terminal
Datei Bearbeiten Darstellung Suchen Terminal Hilfe
innovus 102> timeDesign -postCTS
*** timeDesign #2 [begin] : totSession cpu/real = 0:27:44.2/0:39:09.2 (0.7), mem = 4138.6M
All nets are already routed correctly.
timeDesign Summary
Density: 25.488%
Routing Overflow: 0.01% H and 0.01% V
Total CPU time: 7.06 sec
Total Real time: 8.0 sec
Total Memory Usage: 4118.808594 Mbytes
*** timeDesign #2 [finish] : cpu/real = 0:00:07.1/0:00:07.7 (0.9), totSession cpu/real = 0:27:51.2/0:39:16.9 (0.7), mem = 4118.8M
innovus 103> innovus 103>



3.7.6 Routing

In this stage, global and detailed routing was performed to connect all nets in the design properly. After routing, filler cells were added to fill empty spaces in the standard-cell rows and maintain design continuity. Finally, post-route timing analysis was done for both setup and hold checks, and routing reports were generated.

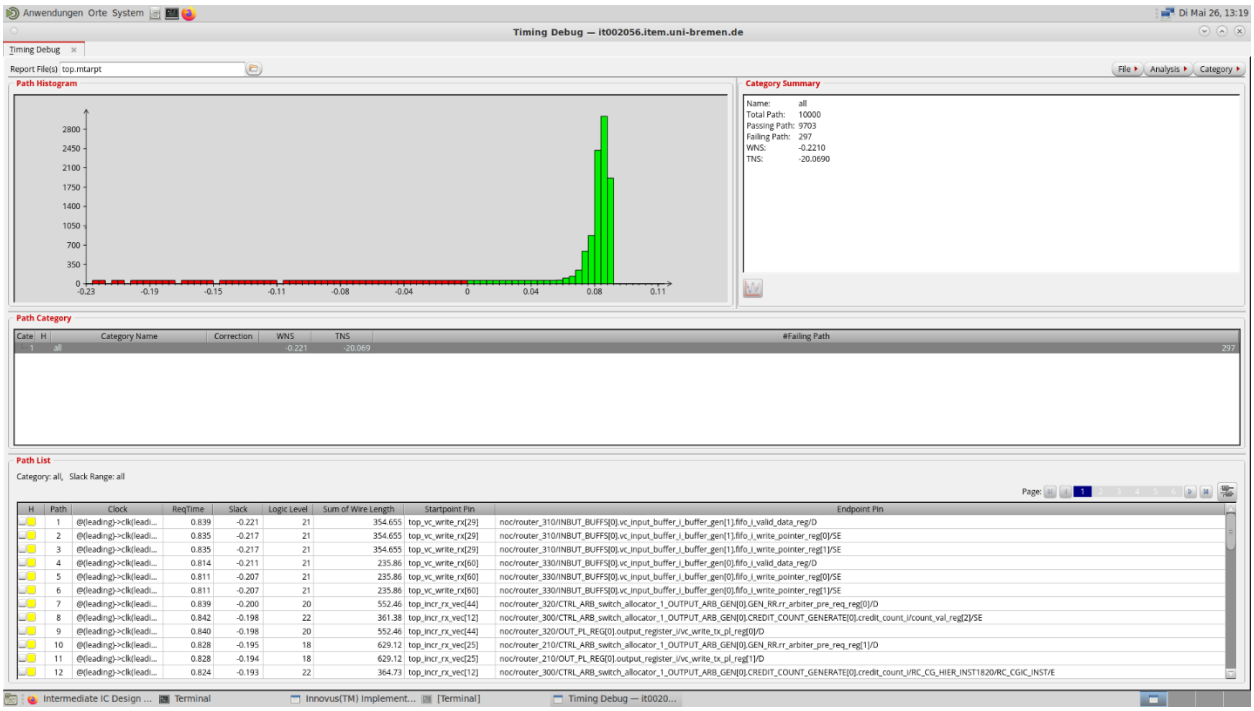




3.7.7 Optimization

In this step, we fixed the hold timing violations that were found after routing. First, the filler cells were removed, and the design was optimized to improve the hold timing. After the optimization was finished, the filler cells were added back.

Then, timing analysis was run again to check both setup and hold timing. Power and area reports were also generated. The timing debugger was used to find and inspect any paths that still had timing problems.



H	Path	Clock	ReqTime	Slack	Logic Level	Sum of Wire Length	Startpoint Pin	Endpoint Pin
	1	@{leading}->clk{leadi...	0.839	-0.221	21	354.655	top_vc_write_rx[29]	noc/router_310/INBUT_BUFFS[0].vc_input_buffer_i_buffer_gen[1].fifo_i_valid_data_reg/D
	2	@{leading}->clk{leadi...	0.835	-0.217	21	354.655	top_vc_write_rx[29]	noc/router_310/INBUT_BUFFS[0].vc_input_buffer_i_buffer_gen[1].fifo_i_write_pointer_reg[0]/SE
	3	@{leading}->clk{leadi...	0.835	-0.217	21	354.655	top_vc_write_rx[29]	noc/router_310/INBUT_BUFFS[0].vc_input_buffer_i_buffer_gen[1].fifo_i_write_pointer_reg[1]/SE
	4	@{leading}->clk{leadi...	0.814	-0.211	21	235.86	top_vc_write_rx[60]	noc/router_330/INBUT_BUFFS[0].vc_input_buffer_i_buffer_gen[0].fifo_i_valid_data_reg/D
	5	@{leading}->clk{leadi...	0.811	-0.207	21	235.86	top_vc_write_rx[60]	noc/router_330/INBUT_BUFFS[0].vc_input_buffer_i_buffer_gen[0].fifo_i_write_pointer_reg[0]/SE
	6	@{leading}->clk{leadi...	0.811	-0.207	21	235.86	top_vc_write_rx[60]	noc/router_330/INBUT_BUFFS[0].vc_input_buffer_i_buffer_gen[0].fifo_i_write_pointer_reg[1]/SE
	7	@{leading}->clk{leadi...	0.839	-0.200	20	552.46	top_incr_rx_vec[44]	noc/router_320/CTRL_ARB_switch_allocator_1_OUTPUT_ARB_GEN[0].GEN_RR.rr_arbiter_pre_req_reg[0]/D
	8	@{leading}->clk{leadi...	0.842	-0.198	22	361.38	top_incr_rx_vec[12]	noc/router_300/CTRL_ARB_switch_allocator_1_OUTPUT_ARB_GEN[0].CREDIT_COUNT_GENERATE[0].credit_count_i/count_val_reg[2]/SE
	9	@{leading}->clk{leadi...	0.840	-0.198	20	552.46	top_incr_rx_vec[44]	noc/router_320/OUT_PL_REG[0].output_register_i/vc_write_tx_pl_reg[0]/D
	10	@{leading}->clk{leadi...	0.828	-0.195	18	629.12	top_incr_rx_vec[25]	noc/router_210/CTRL_ARB_switch_allocator_1_OUTPUT_ARB_GEN[0].GEN_RR.rr_arbiter_pre_req_reg[1]/D
	11	@{leading}->clk{leadi...	0.828	-0.194	18	629.12	top_incr_rx_vec[25]	noc/router_210/OUT_PL_REG[0].output_register_i/vc_write_tx_pl_reg[1]/D
	12	@{leading}->clk{leadi...	0.824	-0.193	22	364.73	top_incr_rx_vec[12]	noc/router_300/CTRL_ARB_switch_allocator_1_OUTPUT_ARB_GEN[0].CREDIT_COUNT_GENERATE[0].credit_count_i/RC_CG_HIER_INST1820/RC_CGIC_INST/E